

Week 3: 탐색 최적화 - Iterative Deepening, Transposition Table, PVS

목차

1. 복습: Alpha-Beta Pruning
 2. Iterative Deepening (반복 심화)
 3. Zobrist Hashing
 4. Transposition Table (치환표)
 5. Move Ordering과 TT
 6. TT Cutoff
 7. Principal Variation Search (PVS)
 8. 누적 성능 향상 비교
 9. 핵심 정리 및 다음 주 예고
-

1. 복습: Alpha-Beta Pruning

1.1 지난 주 핵심 내용

Minimax 알고리즘의 한계:

- 모든 경우의 수를 탐색 (b^d 노드)
- 체스에서 $b=35$, $d=10$ 이면 약 2.7조 노드
- 실용적 시간 내 깊은 탐색 불가능

Alpha-Beta Pruning의 핵심:

- 최선의 경우: 탐색 노드 수를 $b^d \rightarrow b^{(d/2)}$ 로 감소
- Alpha: 최대화 플레이어가 보장할 수 있는 최소 점수
- Beta: 최소화 플레이어가 보장할 수 있는 최대 점수
- $\text{Alpha} \geq \text{Beta}$ 일 때 가지치기 발생

의사코드 복습:

```
def alpha_beta(node, depth, alpha, beta, maximizing):
    if depth == 0 or node.is_terminal():
        return evaluate(node)

    if maximizing:
        value = -INF
        for child in node.children():
            value = max(value, alpha_beta(child, depth-1, alpha, beta,
False))
            alpha = max(alpha, value)
            if alpha >= beta:
                break # Beta cutoff
        return value
```

```

else:
    value = INF
    for child in node.children():
        value = min(value, alpha_beta(child, depth-1, alpha, beta,
True))
        beta = min(beta, value)
        if beta <= alpha:
            break # Alpha cutoff
    return value

```

핵심 성능 요인:

- Move Ordering: 좋은 수를 먼저 탐색할수록 가지치기 증가
- 최선 순서: 첫 번째 자식이 항상 최선 $\rightarrow b^{(d/2)}$ 달성
- 무작위 순서: 평균적으로 $b^{(3d/4)}$ 정도

오늘의 목표: 지난 주 Alpha-Beta의 성능을 더욱 극대화하는 세 가지 핵심 기법을 배웁니다:

1. **Iterative Deepening:** 제한된 시간 내에 최적의 깊이까지 탐색
2. **Transposition Table:** 중복 계산 제거 + 최고의 Move Ordering 제공
3. **Principal Variation Search:** Null window를 활용한 초고속 탐색

2. Iterative Deepening (반복 심화)

2.1 문제 상황

고정 깊이 탐색의 딜레마:

시나리오 1: 깊이를 너무 낮게 설정 (depth=4)

- 탐색 시간: 0.1초
- 남은 시간: 1분 50초
- 문제: 시간이 충분한데도 얕은 탐색만 수행 $\rightarrow$ 약한 수 선택

시나리오 2: 깊이를 너무 높게 설정 (depth=10)

- 탐색 시간: 5분
- 시간 제한: 2분
- 문제: 시간 초과로 실격

질문: 최적의 깊이를 어떻게 미리 알 수 있을까?

- 분기 인수(branching factor)는 매 수마다 변화
- 보드 상태에 따라 복잡도가 다름
- 정답: 미리 알 수 없다!

2.2 Iterative Deepening의 해결책

핵심 아이디어: 깊이를 1부터 시작하여 시간이 허락하는 한 깊이를 1씩 증가시키며 탐색합니다.

```
def iterative_deepening(board, time_limit):
    best_move = None
    start_time = current_time()

    for depth in range(1, MAX_DEPTH):
        if elapsed_time(start_time) > time_limit * 0.9:
            break # 시간 제한의 90%에 도달하면 중단

        score, move = alpha_beta(board, depth, -INF, INF)
        best_move = move # 더 깊은 탐색 결과로 갱신

    return best_move
```

의사코드 설명:

1. depth=1부터 시작
2. Alpha-Beta로 현재 깊이 탐색
3. 결과를 best_move에 저장
4. 시간이 충분하면 depth를 1 증가
5. 시간 제한에 근접하면 중단하고 마지막 best_move 반환

2.3 시간 복잡도 분석

의문: 같은 보드를 여러 번 탐색하면 시간 낭비 아닌가?

수학적 증명:

- 분기 인수 $b=35$, 목표 깊이 $d=6$ 이라고 가정
- 각 깊이별 노드 수 (Alpha-Beta 최선의 경우):
 - depth=1: $b^{(1/2)} = 35^{0.5} \approx 6$
 - depth=2: $b^{(2/2)} = 35^1 = 35$
 - depth=3: $b^{(3/2)} = 35^{1.5} \approx 207$
 - depth=4: $b^{(4/2)} = 35^2 = 1,225$
 - depth=5: $b^{(5/2)} = 35^{2.5} \approx 7,218$
 - depth=6: $b^{(6/2)} = 35^3 = 42,875$

총 노드 수:

- Iterative Deepening: $6 + 35 + 207 + 1,225 + 7,218 + 42,875 = 51,566$
- 단일 탐색 (depth=6): 42,875

오버헤드: $(51,566 - 42,875) / 42,875 = 20\%$

결론:

- 이론적으로 약 20% 오버헤드
- 하지만 실전에서는 다음 장점으로 상쇄:
 1. 시간 관리 자동화
 2. Move Ordering 향상 (아래에서 설명)
 3. 안정성: 항상 유효한 수를 반환

2.4 시간 관리 전략

게임 단계별 시간 배분:

ATAXX 게임 기준 (총 시간 120초):

- 초반 (남은 시간 > 60초): 빠른 수 (50ms/턴)
- 중반 (60초 ≥ 남은 시간 > 20초): 표준 수 (150ms/턴)
- 종반 (남은 시간 ≤ 20초): 빠른 수 (10ms/턴)

```
def calculate_time_limit(my_time, turn_count):
    """턴당 시간 배분 계산"""
    if my_time > 60000: # 60초 이상
        return 50 # 50ms
    elif my_time > 20000: # 20초 이상
        return 150 # 150ms
    else:
        return 10 # 10ms (안전 마진)
```

적응형 시간 관리:

```
def adaptive_time_limit(my_time, moves_left_estimate):
    """남은 수 예측 기반 시간 배분"""
    # 안전 마진 20% 확보
    available_time = my_time * 0.8

    # 남은 수가 많으면 보수적, 적으면 공격적
    time_per_move = available_time / max(moves_left_estimate, 5)

    # 최소/최대 제한
    return max(10, min(time_per_move, 500))
```

2.5 Move Ordering에 대한 힌트

Iterative Deepening의 숨겨진 보너스:

이전 깊이(depth=5)의 탐색 결과:

- 최선의 수: e2e3 (점수: +15)

다음 깊이(depth=6) 탐색 시:

- e2e3를 가장 먼저 탐색
- 높은 확률로 여전히 최선의 수
- Alpha-Beta의 가지치기 효율 극대화

성능 향상:

- Iterative Deepening + Move Ordering: **+50 Elo**

- 오버헤드를 상쇄하고도 남는 이득

3. Zobrist Hashing

3.1 보드 상태의 고유 식별 문제

왜 해싱이 필요한가?

ATAXX 7x7 보드:

- 각 칸: 빈칸(0), 흑돌(1), 백돌(2), 벽(3) → 4가지 상태
- 전체 경우의 수: $4^{49} \approx 5.6 \times 10^{29}$

딕셔너리 키로 사용하려면:

- 보드 전체를 문자열로 변환: "11200000..." (49자)
 - 메모리 낭비
 - 비교 연산 느림 ($O(49)$)
- 더 나은 방법: 정수 하나로 표현 ($O(1)$ 비교)

3.2 Zobrist Hashing 원리

1970년 Albert Zobrist가 발명한 체스 해싱 기법

핵심 아이디어:

1. 각 (위치, 돌 종류) 조합마다 랜덤한 64비트 정수 할당
2. 보드 상태 = 모든 돌의 해시값을 XOR한 결과

초기화:

```
import random

# Zobrist 테이블 생성
zobrist = {}
for x in range(7):
    for y in range(7):
        for piece in [EMPTY, BLACK, WHITE, WALL]:
            zobrist[(x, y, piece)] = random.getrandbits(64)

# 초기 보드의 해시값 계산
def compute_hash(board):
    h = 0
    for x in range(7):
        for y in range(7):
            piece = board[x][y]
            h ^= zobrist[(x, y, piece)]
    return h
```

증분 업데이트 (핵심 장점):

```
# 돌을 (x, y)에서 이동
def update_hash(hash_value, x, y, old_piece, new_piece):
    # XOR의 성질: A ^ A = 0, A ^ 0 = A
    hash_value ^= zobrist[(x, y, old_piece)] # 이전 상태 제거
    hash_value ^= zobrist[(x, y, new_piece)] # 새 상태 추가
    return hash_value
```

시간 복잡도:

- 전체 보드 해싱: $O(49) = O(n^2)$
- 증분 업데이트: $O(1)$
- Undo도 동일한 XOR로 가능: `hash ^= zobrist[...] ^= zobrist[...]`

3.3 XORShift 기반 해시 함수

더 빠른 해시 생성: XORShift

Python의 `random.getrandbits()`는 느릴 수 있습니다. XORShift는 간단하면서도 분포가 좋은 의사난수 생성기입니다.

```
def xorshift64(x):
    """64비트 XORShift PRNG"""
    x ^= (x << 13) & 0xFFFFFFFFFFFFFFFF
    x ^= (x >> 7)
    x ^= (x << 17) & 0xFFFFFFFFFFFFFFFF
    return x

# Zobrist 테이블 초기화 (XORShift 사용)
seed = 123456789
zobrist = {}
for x in range(7):
    for y in range(7):
        for piece in range(4):
            seed = xorshift64(seed)
            zobrist[(x, y, piece)] = seed
```

장점:

- 매우 빠름 (비트 연산만 사용)
- 재현 가능 (같은 seed → 같은 테이블)
- 충돌 확률 낮음

3.4 해시 충돌 처리

생일 문제 (Birthday Paradox):

- 64비트 해시: $2^{64} \approx 1.8 \times 10^{19}$
- 약 2^{32} (43억) 개의 상태를 저장하면 충돌 확률 50%
- 게임 AI에서는 수백만 상태 정도만 저장 → 충돌 확률 극히 낮음

충돌 발생 시 전략:

1. **무시:** 충돌 확률이 낮아서 게임 결과에 미미한 영향
2. **검증:** 해시가 같으면 실제 보드도 비교 (완벽하지만 느림)
3. **Replace 정책:** 새로운 값으로 덮어쓰기

실전에서는:

- 대부분 무시 전략 사용
- 깊이(depth) 정보를 함께 저장하여 더 깊은 탐색 결과 우선

4. Transposition Table (치환표)

4.1 Transposition(치환)이란?

같은 보드, 다른 경로:

초기 보드

```

  ↙   ↘
수 A   수 B
  ↘   ↙
동일한 보드!
```

ATAXX 예시:

- 경로 1: (1,1)→(1,2) 후 (2,1)→(3,1)
- 경로 2: (2,1)→(3,1) 후 (1,1)→(1,2)
- 결과: 동일한 보드 상태

일반적인 Alpha-Beta:

- 두 경로에서 모두 탐색 수행
- 중복 계산으로 시간 낭비

Transposition Table (TT):

- 보드 상태별로 탐색 결과 저장
- 같은 보드 재방문 시 저장된 결과 재사용

4.2 TT 엔트리 구조

저장할 정보:

```
class TTEntry:
    def __init__(self, best_move, flag, depth, value):
        self.best_move = best_move # 최선의 수
        self.flag = flag           # PV_NODE, CUT_NODE, ALL_NODE
        self.depth = depth        # 탐색 깊이
        self.value = value        # 평가값
```

Flag의 의미:**1. PV_NODE (Exact Value):**

- $\alpha < \text{value} < \beta$
- 정확한 값
- 신뢰도 최고

2. CUT_NODE (Lower Bound):

- $\text{value} \geq \beta$
- Beta cutoff 발생
- 실제 값은 이 값 이상

3. ALL_NODE (Upper Bound):

- $\text{value} \leq \alpha$
- Alpha cutoff 발생
- 실제 값은 이 값 이하

4.3 TT를 사용한 Alpha-Beta

```
# 전역 Transposition Table
tt = {} # {hash: TTEntry}

def alpha_beta_tt(board, depth, alpha, beta):
    # 1. TT 조회
    board_hash = board.hash()
    if board_hash in tt:
        entry = tt[board_hash]
        # Move Ordering을 위해 best_move는 항상 사용
        # (Cutoff는 나중에 다룸)

    # 2. 종료 조건
    if depth == 0 or board.is_terminal():
        return evaluate(board), None

    # 3. 수 생성 및 정렬
    moves = board.legal_moves()

    # Move Ordering: TT의 best_move를 맨 앞으로
    if board_hash in tt and tt[board_hash].best_move:
        best_move = tt[board_hash].best_move
        if best_move in moves:
            moves.remove(best_move)
            moves.insert(0, best_move)

    # 4. 탐색
    best_value = -INF
    best_move = None
```



```

for move in moves:
    board.make_move(move)
    value, _ = alpha_beta_tt(board, depth-1, -beta, -alpha)
    value = -value
    board.undo_move()

    if value > best_value:
        best_value = value
        best_move = move

    alpha = max(alpha, value)
    if alpha >= beta:
        # Beta cutoff
        break

# 5. TT 저장
if best_value <= alpha_original:
    flag = ALL_NODE
elif best_value >= beta:
    flag = CUT_NODE
else:
    flag = PV_NODE

tt[board_hash] = TTEntry(best_move, flag, depth, best_value)

return best_value, best_move

```

4.4 TT 메모리 관리

메모리 한계:

- 파이썬 딕셔너리: 메모리 제한 없이 계속 증가 가능
- 실전: 수백 MB ~ 수 GB까지 증가 가능

Replace 전략:

1. Always Replace:

- 항상 새 값으로 덮어쓰기
- 구현 간단
- 최신 정보 우선

2. Depth-Preferred:

```

if board_hash not in tt or tt[board_hash].depth <= depth:
    tt[board_hash] = TTEntry(...)

```

- 더 깊은 탐색 결과 우선
- 더 정확한 정보 보존

3. Two-Tier (Age + Depth):

```
if board_hash not in tt or \
    tt[board_hash].age < current_search or \
    tt[board_hash].depth <= depth:
    tt[board_hash] = TTEntry(...)
```

- 현재 탐색의 결과 우선
- 깊이도 고려

크기 제한:

```
MAX_TT_SIZE = 10_000_000 # 1천만 엔트리

if len(tt) > MAX_TT_SIZE:
    # 가장 오래된 절반 삭제 (LRU 근사)
    tt.clear() # 또는 더 정교한 정책
```

5. Move Ordering과 TT

5.1 Move Ordering의 중요성 재강조

Alpha-Beta의 성능 = Move Ordering의 품질

순서 품질	탐색 노드 수	예시 (b=35, d=6)
완벽한 순서	$b^{(d/2)}$	$35^3 = 42,875$
무작위 순서	$b^{(3d/4)}$	$35^{4.5} \approx 253,000$
최악의 순서	b^d	$35^6 \approx 1.8\text{억}$

차이: 완벽한 순서는 무작위 대비 **6배** 빠름

5.2 TT를 활용한 Move Ordering

Iterative Deepening + TT의 시너지:

1. Depth=5 탐색:

- 최선의 수: e2e3
- TT에 저장: `tt[hash] = TTEntry(best_move="e2e3", ...)`

2. Depth=6 탐색:

- TT 조회: "이전에 e2e3가 최선이었음"
- e2e3를 맨 먼저 탐색
- 높은 확률로 여전히 최선 → 즉시 alpha 상승

- 나머지 수들은 높은 alpha로 인해 가지치기

구현:

```
moves = board.legal_moves()

# TT에서 best_move 가져오기
if board_hash in tt and tt[board_hash].best_move:
    tt_move = tt[board_hash].best_move
    if tt_move in moves:
        # TT move를 맨 앞으로
        moves.remove(tt_move)
        moves.insert(0, tt_move)

# 추가 휴리스틱 정렬 (선택사항)
# - MVV-LVA (Most Valuable Victim - Least Valuable Attacker)
# - 위치 점수 (중앙 우선)
# - 킬러 휴리스틱
```

5.3 성능 측정

실험 설정:

- 베이스라인: Iterative Deepening + Alpha-Beta (Move Ordering 없음)
- 개선: ID + AB + TT Move Ordering

결과:

- **+50 Elo 향상**
- 같은 시간에 평균 1~2 depth 더 깊이 탐색
- Branching factor가 낮은 게임일수록 효과 큼

Elo 해석:

- +50 Elo = 승률 약 57% (1000게임 중 570승)
- 누적 향상: 안정적인 성능 증가

6. TT Cutoff

6.1 TT Cutoff의 아이디어

질문: TT에 저장된 값을 탐색 없이 바로 사용할 수 있을까?

조건:

1. 저장된 깊이 $\geq$ 현재 필요한 깊이
2. Flag에 따라 alpha/beta와 비교

Cutoff 조건:

```

if board_hash in tt:
    entry = tt[board_hash]

    # 깊이가 충분한가?
    if entry.depth >= depth:

        # Flag에 따른 cutoff
        if entry.flag == PV_NODE:
            # 정확한 값 → 바로 반환
            return entry.value, entry.best_move

        elif entry.flag == CUT_NODE:
            # Lower bound: value >= entry.value
            if entry.value >= beta:
                return entry.value, entry.best_move

        elif entry.flag == ALL_NODE:
            # Upper bound: value <= entry.value
            if entry.value <= alpha:
                return entry.value, entry.best_move

```

6.2 깊이 조건의 중요성

왜 저장된 깊이가 더 깊어야 하는가?

예시:

- 이전 탐색: depth=3, value=+10
- 현재 탐색: depth=5

문제:

- depth=3의 결과는 3수 앞까지만 본 것
- depth=5는 5수 앞을 봐야 함
- 얕은 결과를 사용하면 전략적 실수 발생

안전한 사용:

```

if entry.depth >= depth:
    # 저장된 결과가 더 깊거나 같음 → 신뢰 가능
    ...

```

6.3 실전 성능 - 의외의 결과

실험 결과:

- ATAXX 게임 기준: ± 0 Elo
- 즉, 성능 향상 없음!

원인 분석:

1. 얕은 탐색에서의 오버헤드:

- TT 조회/저장: 해시 계산, 딕셔너리 접근
- 깊이 4~6: 탐색 자체가 빠름
- TT 오버헤드 > 절약한 시간

2. 깊이 조건의 엄격함:

- Iterative Deepening: 매번 depth 증가
- 이전 depth의 결과는 항상 `entry.depth < depth`
- Cutoff 발생 확률 낮음

3. 게임 특성:

- ATAXX: Transposition 빈도가 중간 정도
- 체스/장기: Transposition 매우 빈번 → 효과 큼

언제 유용한가?

- 깊은 탐색 (depth > 10)
- Transposition이 빈번한 게임
- 메모리가 충분한 환경

권장 사항:

- 학습 목적: 구현하고 실험해보기
- 실전 경쟁: 시간 관리에 집중, TT Cutoff는 선택사항

7. Principal Variation Search (PVS)

7.1 PVS의 핵심 통찰

관찰:

- Alpha-Beta에서 첫 번째 자식이 최선일 확률이 높음 (Move Ordering 덕분)
- 나머지 자식들은 대부분 더 나쁜 수

아이디어:

1. 첫 번째 자식: 정상적으로 탐색 (full window)
2. 나머지 자식들: "이 수가 첫 번째보다 나은가?"만 확인 (null window)
 - Null window로 빠르게 확인
 - 만약 더 나으면 → full window로 재탐색

비유:

- Full window: 정밀 측정 (시간 많이 걸림)
- Null window: 참/거짓 확인 (빠름)

7.2 Null Window 탐색

Null Window란?

```
# Full window
value = alpha_beta(node, depth, alpha, beta)
# alpha와 beta 사이에 실제 값이 있는지 탐색

# Null window
value = alpha_beta(node, depth, alpha, alpha+1)
# value > alpha이지만 확인 (Yes/No 질문)
```

왜 빠른가?

- Window가 좁을수록 가지치기 빈번
- alpha=10, beta=11: 11 이상이면 즉시 cutoff
- 평균적으로 2~3배 빠름

7.3 PVS 의사코드

```
def pvs(board, depth, alpha, beta, is_first_child=True):
    # 종료 조건
    if depth == 0 or board.is_terminal():
        return evaluate(board), None

    moves = board.legal_moves()
    # Move Ordering (TT 활용)
    moves = order_moves(moves, board)

    best_value = -INF
    best_move = None

    for i, move in enumerate(moves):
        board.make_move(move)

        if i == 0:
            # 첫 번째 자식: Full window
            value, _ = pvs(board, depth-1, -beta, -alpha, True)
            value = -value
        else:
            # 나머지: Null window로 확인
            value, _ = pvs(board, depth-1, -alpha-1, -alpha, False)
            value = -value

            # Null window 실패 (value > alpha) → 재탐색
            if alpha < value < beta:
                value, _ = pvs(board, depth-1, -beta, -value, True)
                value = -value

        board.undo_move()

        if value > best_value:
```

```

        best_value = value
        best_move = move

    alpha = max(alpha, value)
    if alpha >= beta:
        break # Beta cutoff

    return best_value, best_move

```

7.4 PVS 동작 예시

보드 상태:

- 합법적 수: [A, B, C, D, E]
- Move Ordering 후: [A, B, C, D, E] (A가 TT의 best move)

탐색 과정:

1. 수 A (첫 번째):

- Window: [alpha=-10, beta=20]
- 결과: value=15
- alpha 갱신: alpha=15

2. 수 B:

- Null window: [15, 16]
- 결과: value=14 (alpha보다 작음)
- 재탐색 불필요 → 빠르게 스킵

3. 수 C:

- Null window: [15, 16]
- 결과: value=17 (alpha보다 큼!)
- 재탐색: [15, 20]
- 최종 value=17
- alpha 갱신: alpha=17

4. 수 D:

- Null window: [17, 18]
- 결과: value=12
- 스킵

5. 수 E:

- Null window: [17, 18]
- 결과: value=16
- 스킵

통계:

- Full window 탐색: 2회 (A, C의 재탐색)
- Null window 탐색: 4회 (B, C의 첫 시도, D, E)
- 일반 Alpha-Beta였다면: 5회 full window

7.5 재탐색(Re-search)의 이해

재탐색이 발생하는 경우:

```
if alpha < value < beta:
    # Null window 결과가 alpha와 beta 사이
    # → 정확한 값 필요 → Full window 재탐색
```

언제 발생하나?

- Move Ordering이 틀렸을 때
- 첫 번째 수가 최선이 아닐 때

성능 영향:

- Move Ordering이 좋으면: 재탐색 거의 없음 (5% 미만)
- Move Ordering이 나쁘면: 재탐색 빈번 (50% 이상) → 오히려 느려질 수 있음

결론:

- **PVS는 Move Ordering과의 조합이 핵심**
- TT Move Ordering + PVS = 최강의 조합

7.6 PVS 성능

실험 결과:

- 베이스라인: ID + AB + TT Move Ordering
- 개선: 위에 + PVS
- 성능 향상: **+50 Elo**

효과:

- 탐색 노드 30~40% 감소
- 같은 시간에 1 depth 더 깊이 탐색 가능

주의사항:

- Move Ordering이 나쁘면 역효과
- 구현 복잡도 증가 (재탐색 로직)

8. 누적 성능 향상 비교

8.1 단계별 Elo 변화

기법	누적 Elo	단계 Elo 변화	주요 효과
Minimax (Week 1)	0	-	기본 탐색 (b^d)
Alpha-Beta (Week 2)	+200	+200	탐색 노드 $b^{(3d/4)} \sim b^{(d/2)}$
Iterative Deepening	+250	+50	시간 관리 + ID
TT Move Ordering	+300	+50	가지치기 효율 극대화
TT Cutoff	+300	± 0	얕은 탐색에서 오버헤드
PVS	+350	+50	Null window 최적화

해석:

- Alpha-Beta: 가장 큰 도약 (+200 Elo)
- 이후 기법들: 각각 +50 Elo씩 꾸준한 향상
- TT Cutoff: ATAXX에서는 효과 미미 (게임마다 다름)

8.2 탐색 효율 비교

동일 시간(100ms) 내 탐색 깊이:

알고리즘	평균 깊이	탐색 노드 수
Minimax	4	1,500,000
Alpha-Beta	6	250,000
+ ID + TT MO	7	100,000
+ PVS	8	70,000

깊이 1 증가의 의미:

- 한 수 더 앞을 내다봄
- 전략적 실수 감소
- 승률 크게 향상

8.3 게임별 효과 차이

게임	Alpha-Beta	TT Move Ordering	PVS	비고
ATAXX	+200	+50	+50	중간 복잡도
체스	+300	+80	+70	Transposition 빈번
오델로	+150	+40	+40	분기 인수 낮음
바둑	+50	+10	+10	분기 인수 너무 높음 ($b \sim 250$)

관찰:

- Transposition이 빈번한 게임 → TT 효과 큼

- 분기 인수가 낮은 게임 → Move Ordering 효과 큼
- 바둑은 너무 복잡 → MCTS로 접근 (다음 주)

8.4 메모리 vs 속도 트레이드오프

Transposition Table 크기별 성능:

TT 크기	메모리	Hit Rate	Elo
없음	0 MB	0%	기준
100만	80 MB	30%	+30
1000만	800 MB	60%	+50
1억	8 GB	75%	+55

권장:

- 실전 대화: 1000만~5000만 엔트리 (0.8~4 GB)
- 학습용: 100만 엔트리 (80 MB)

9. 핵심 정리 및 다음 주 예고

9.1 Week 3 핵심 요약

1. Iterative Deepening (반복 심화)

- 깊이 1부터 시작, 시간 내에 최대한 깊이 탐색
- 약 20% 오버헤드이지만 안정성과 Move Ordering 보너스로 상쇄
- 시간 관리 자동화

2. Zobrist Hashing

- 보드 상태를 64비트 정수로 표현
- XOR 기반 O(1) 증분 업데이트
- Transposition Table의 핵심 기반

3. Transposition Table

- 중복 계산 제거
- **Move Ordering**: TT best move 우선 탐색 (+50 Elo)
- **TT Cutoff**: 얇은 탐색에서는 ± 0 Elo (게임마다 다름)

4. Principal Variation Search

- 첫 수: Full window
- 나머지: Null window → 필요시 재탐색
- Move Ordering과 결합 시 +50 Elo

종합 성능:

- Week 2 (Alpha-Beta) 대비 **+150 Elo 향상**

- 탐색 깊이 1~2 depth 증가
- 메모리 사용량 증가 (TT)

9.2 구현 시 주의사항

1. 시간 관리:

```
# 안전 마진 확보
time_limit = calculate_time_limit(my_time) * 0.9

# 매 루트 노드마다 시간 체크
if current_time() - start_time > time_limit:
    break
```

2. Zobrist 초기화:

```
# 게임 시작 시 한 번만 초기화
zobrist = initialize_zobrist()

# 매 수마다 증분 업데이트
hash = update_hash(hash, move)
```

3. TT Replace 정책:

```
# Depth-preferred
if hash not in tt or tt[hash].depth <= depth:
    tt[hash] = TTEntry(...)
```

4. PVS 재탐색 처리:

```
# Null window 실패 시 재탐색 필수
if alpha < value < beta:
    value = -pvs(..., -beta, -value)
```

9.3 디버깅 팁

1. TT 검증:

```
# TT에서 가져온 수가 합법적인지 확인
if tt_move in legal_moves:
    ...
else:
    print(f"WARNING: TT move {tt_move} is illegal!")
```

2. 시간 초과 방지:

```
# Iterative Deepening 종료 조건
if elapsed_time > time_limit * 0.9:
    break # 90%에서 중단
```

3. PVS 재탐색 통계:

```
# 디버깅용: 재탐색 비율 출력
re_searches / total_searches
# 이상적: < 10%
```

9.4 다음 주 예고: Monte Carlo Tree Search (MCTS)

Alpha-Beta의 한계:

- 분기 인수가 높은 게임 (바둑 $b \sim 250$)
- 평가 함수가 부정확한 게임
- 깊이 제한으로 인한 전략적 손실

MCTS의 등장:

- 2006년, 바둑 AI의 혁명
- AlphaGo의 핵심 알고리즘
- 평가 함수 없이도 강력한 성능

Week 4 내용:

1. MCTS 기본 원리

- Selection, Expansion, Simulation, Backpropagation
- UCB1 (Upper Confidence Bound)

2. 구현

- 트리 구조
- 시뮬레이션 정책
- 시간 관리

3. 최적화

- RAVE (Rapid Action Value Estimation)
- Progressive Widening
- Parallelization

4. MCTS vs Alpha-Beta 비교

- 게임별 적합성

- 하이브리드 접근

준비 사항:

- Alpha-Beta 코드 완성
- TT 구현 숙지 (MCTS에서도 사용)
- 트리 자료구조 복습

9.5 추가 학습 자료

논문:

1. Zobrist (1970): "A New Hashing Method with Application for Game Playing"
2. Schaeffer (1989): "The History Heuristic and Alpha-Beta Search Enhancements"
3. Marsland & Campbell (1982): "Parallel Search of Strongly Ordered Game Trees"

온라인 자료:

- Chess Programming Wiki: <https://www.chessprogramming.org/>
- Infosm Blog: <https://infosm.github.io/> (이번 주 참고 자료)
- Alpha-Beta 시뮬레이터: https://inst.eecs.berkeley.edu/~cs61b/fa14/ta-materials/apps/ab_tree_practice/

실습 문제:

- Week 3 Baekjoon 문제 8개
- ALPHANO 리더보드 제출

부록: 전체 코드 예시

A.1 Iterative Deepening + TT + PVS 통합

```
import time

# Zobrist 초기화
def init_zobrist():
    zobrist = {}
    seed = 123456789
    for x in range(7):
        for y in range(7):
            for piece in range(4):
                seed = xorshift64(seed)
                zobrist[(x, y, piece)] = seed
    return zobrist

def xorshift64(x):
    x ^= (x << 13) & 0xFFFFFFFFFFFFFFFF
    x ^= (x >> 7)
    x ^= (x << 17) & 0xFFFFFFFFFFFFFFFF
    return x

# TT Entry
```

```

class TTEEntry:
    def __init__(self, best_move, flag, depth, value):
        self.best_move = best_move
        self.flag = flag
        self.depth = depth
        self.value = value

# Flag 상수
PV_NODE, CUT_NODE, ALL_NODE = 0, 1, 2

# 전역 TT
tt = {}

# PVS with TT
def pvs(board, depth, alpha, beta, use_null_window):
    alpha_original = alpha

    # TT 조회
    board_hash = board.hash()
    tt_move = None
    if board_hash in tt:
        entry = tt[board_hash]
        tt_move = entry.best_move

    # TT Cutoff (선택사항)
    if entry.depth >= depth:
        if entry.flag == PV_NODE:
            return entry.value, entry.best_move
        elif entry.flag == CUT_NODE and entry.value >= beta:
            return entry.value, entry.best_move
        elif entry.flag == ALL_NODE and entry.value <= alpha:
            return entry.value, entry.best_move

    # 종료 조건
    if depth == 0 or board.is_terminal():
        return board.evaluate(), None

    # 수 생성 및 정렬
    moves = board.legal_moves()
    if tt_move and tt_move in moves:
        moves.remove(tt_move)
        moves.insert(0, tt_move)

    # 탐색
    best_value = -float('inf')
    best_move = None

    for i, move in enumerate(moves):
        board.make_move(move)

        if i == 0:
            # 첫 번째 수: Full window
            value, _ = pvs(board, depth-1, -beta, -alpha, False)
            value = -value

```

```

    else:
        # Null window
        value, _ = pvs(board, depth-1, -alpha-1, -alpha, True)
        value = -value

        # 재탐색
        if alpha < value < beta:
            value, _ = pvs(board, depth-1, -beta, -value, False)
            value = -value

    board.undo_move()

    if value > best_value:
        best_value = value
        best_move = move

    alpha = max(alpha, value)
    if alpha >= beta:
        break

# TT 저장
if best_value <= alpha_original:
    flag = ALL_NODE
elif best_value >= beta:
    flag = CUT_NODE
else:
    flag = PV_NODE

tt[board_hash] = TTEntry(best_move, flag, depth, best_value)

return best_value, best_move

# Iterative Deepening
def iterative_deepening(board, time_limit):
    start_time = time.time()
    best_move = None

    for depth in range(1, 50):
        if (time.time() - start_time) * 1000 > time_limit * 0.9:
            break

        value, move = pvs(board, depth, -float('inf'), float('inf'),
False)
        if move:
            best_move = move

    return best_move

```

A.2 시간 관리

```
def calculate_time_limit(my_time, turn_count):  
    """적응형 시간 배분"""  
    if my_time > 60000:  
        return 50  
    elif my_time > 20000:  
        return 150  
    else:  
        return 10
```

Week 3 학습 완료!

다음 주에는 Alpha-Beta를 넘어 완전히 새로운 패러다임인 Monte Carlo Tree Search를 배웁니다. ATAXX뿐만 아니라 바둑, 포커 등 다양한 게임에 적용 가능한 강력한 알고리즘입니다!